The Impact of Polysemanticity on the Locality of Neuron-level vs. Layer-level Model Editing

Hongyi Chen
tomchen2005@berkeley.edu

Yubo Fan
yubo@berkeley.edu

Tianhao Qian
lawrence.qian@berkeley.edu

Jie Wen
menger_wen@berkeley.edu

Abstract

Recent work on factual model editing, such as ROME [1] and MEND [2], demonstrates that large language models can be locally edited without full retraining. At the same time, mechanistic interpretability research suggests that many neurons are *polysemantic*, simultaneously encoding multiple features in superposition [5]. This raises a key question: when editing a model, does polysemanticity fundamentally limit how local an edit can be, and does it matter whether we operate at the neuron level or the layer level?

In this draft, we present: (1) an empirical pipeline for measuring neuron-level overlap between pairs of facts using both gradient-based and gradient-free contribution scores; (2) a controlled set of highly correlated and weakly correlated factual pairs for studying locality; and (3) preliminary layer-level editing experiments using ROME on Pythia-6.9B in a resource-constrained environment. We find that editing a high-overlap pair such as *(France, capital→Rome)* causes a large drop (over 70%) in the model's ability to answer a related question (*Eiffel Tower→Paris*), indicating strong entanglement at the layer level.

We have implemented a full neuron-level editing pipeline following the FiNE framework [7], including neuron localization via Eq. (3) and an optimization-based objective for neuron patches. **However, the FiNE-based editing experiments (quantitative locality and global evaluation) have not yet been completed at the time of this draft.** We therefore focus our empirical results on ROME and treat the FiNE results as planned work, describing our experimental design and code infrastructure in detail.

1 Introduction

Large language models (LLMs) store vast amounts of factual knowledge in their parameters. When a fact changes or an error is discovered, it is often desirable to *edit* a model locally rather than retraining from scratch. Factual model editing methods such as ROME [1], MEND [2], and related approaches aim to modify a small number of parameters while preserving global performance and unrelated facts.

In parallel, mechanistic interpretability research has argued that hidden units in deep networks are frequently *polysemantic*: individual neurons participate in multiple unrelated features through superposition [5]. Recent work has started to leverage this structure to identify and manipulate task-specific neurons [6]. Intuitively, if two facts share many of the same underlying neurons, editing one might inevitably affect the other.

This leads to the central question of our project:

How does polysemanticity affect the *locality* of model editing, and does operating at the *neuron level* versus the *layer level* make a difference?

Concretely, we study this question in the context of editing factual associations in autoregressive language models. We:

- Define a neuron-level overlap measure between two facts based on contribution scores for individual neurons.
- Construct a small but controlled set of fact pairs that intuitively share the same object or subject (e.g., "France–Paris" or "Hola–Spanish").
- Run layer-level edits using ROME on Pythia-6.9B and quantify the resulting locality behavior on these pairs.
- Implement a neuron-level editing framework following FiNE [7], including neuron localization (Eq. (3)) and an optimization-based patch objective.

Status of this draft. Our ROME experiments are fully implemented and run end-to-end on all four fact pairs. Our FiNE codebase (localization, editing, evaluation) is also implemented and documented, but due to compute and time limitations, we have not yet completed the large-scale FiNE editing experiments. All FiNE plots and tables described in this draft are therefore *planned only*, and we explicitly mark them as **not yet completed**. The goal of this draft is to obtain feedback on our design and analysis plan before we invest more compute into the FiNE experiments.

2 Background and Related Work

2.1 Factual Model Editing

ROME. The ROME method [1] formulates factual editing as a rank-one update to the MLP weights at a specific layer. Given a subject and relation, it identifies a key layer where the subject representation peaks, then solves a constrained optimization to find an update vector that increases the probability of the new object while regularizing the representation so as not to badly affect other contexts. ROME has been demonstrated on GPT-style models and is known to achieve high edit success with relatively good locality.

MEND and related methods. MEND [2] instead learns a meta-network that predicts weight updates conditioned on the local gradient and context. Other approaches such as MEMIT [3] and KNOWEDIT [4] explore multi-edit regimes, influence-function-based methods, and constraints designed to preserve locality and global performance.

2.2 Polysemanticity and Superposition

Elhage et al. [5] introduced the notion of *toy models of superposition*, showing that when the number of features exceeds the dimensionality of the representation space, neural networks can encode many features in a non-orthogonal basis, leading to polysemantic neurons. This picture is supported by empirical studies in real networks, where many neurons respond to multiple seemingly unrelated concepts.

Recent work on multimodal neurons [6] uses gradient-free contribution scores to identify neurons that strongly contribute to a particular visual or textual concept and demonstrates that editing such neurons can manipulate behavior in a relatively targeted way. This line of work motivates the FiNE framework [7], which explicitly aims to *precisely localize* the neurons responsible for a given fact and to design neuron-level updates that improve locality.

2.3 FiNE: Precise Localization of Memories

FiNE (Precise Localization of Memories in LLMs) proposes a neuron-level localization procedure and an optimization objective that learns small patches on a small set of neurons to edit factual knowledge [7]. Neuron localization is based on Eq. (3) in their paper, which scores a neuron by combining its activation and a projection of the output weights through the unembedding matrix.

Editing is then posed as an optimization over per-neuron vectors Z_j that modify FFN output weights, with an objective balancing edit success, preservation of old behavior, a penalty on over-editing, and regularization.

In this project, we adopt FiNE's perspective and implement both:

- FiNE-style neuron localization (our `compute_fine_contributions` and `locate_neurons_fine` functions), and
- FiNE-style optimization over neuron patches (`FinePatch` and the associated objective).

Important: As of this draft, we have *implemented* these components but have *not yet completed* the full set of FiNE editing experiments and large-scale evaluation.

3 Problem Setup and Metrics

3.1 Fact Pairs and Polysemanticity

We focus on four manually-constructed fact pairs that share either a subject or object:

- Pair 1 (Jobs): **A**: "Steve Jobs founded Apple". **B**: "Steve Jobs worked at Pixar".
- Pair 2 (Gates): **A**: "Bill Gates founded Microsoft". **B**: "Bill Gates is famous for Windows".
- Pair 3 (Paris): **A**: "The capital of France is Paris". **B**: "The Eiffel Tower is located in Paris".
- Pair 4 (Language): **A**: "Hello in Spanish is Hola". **B**: "Hola means Hello in Spanish".

Each pair is used in two roles:

- *Edit fact A*: we rewrite the object (e.g., "France→Rome").
- *Locality fact B*: we measure how the edit affects a related but distinct context (e.g., Eiffel Tower).

At the neuron level, we measure polysemanticity by computing the Jaccard overlap between the top-k neurons for Fact A and Fact B at selected layers:

$$\text{Jaccard}(U_A, U_B) \;=\; \frac{|U_A \cap U_B|}{|U_A \cup U_B|},$$

where U_A and U_B are sets of $(\text{layer}, \text{neuron})$ indices ranked by contribution scores.

3.2 Locality Metrics

Following FiNE and related work, our planned evaluation metrics include:

- **Edit Success**:
$$\mathbb{E}_{(s,r,o^*)\in\mathcal{D}_{\text{edit}}} \mathbf{1}\{\arg\max_y P_{M'}(y|p(s,r)) = o^*\}.$$
- **Locality**:
$$\mathbb{E}_{(s,r)\in\mathcal{D}_{\text{loc}}} \mathbf{1}\{\arg\max_y P_{M'}(y|p(s,r)) = \arg\max_y P_M(y|p(s,r))\},$$
i.e., how often the best answer remains unchanged on unrelated or correlated facts.
- **Over-editing rate**: the fraction of locality prompts where the edited object o^* appears even though it should not.
- **Fluency**: bigram and trigram entropy of generated text, as in FiNE.

In our current draft, the full FiNE-based evaluation pipeline is implemented (Section 6), but **we have not yet run these metrics on large FiNE-edited models**. For ROME, we primarily report pre-/post-edit probabilities for Fact B as a proxy for locality.

4 Methods

4.1 Models and Data

For neuron-level analysis and overlap measurements, we use Pythia-160M, a GPT-NeoX style model with 12 layers, due to its lower memory footprint. For editing experiments, we use Pythia-6.9B, which has 32 layers and better baseline factual performance. Both models are loaded from the EleutherAI Pythia family via `transformers`.

4.2 Neuron Contribution via Activation–Gradient

Our first neuron scoring method follows the standard activation×gradient heuristic widely used in attribution methods. Given a prompt p and a target token t, we:

1. Run a forward pass with labels to compute loss $\mathcal{L}$.
2. Register forward and backward hooks on the FFN output layers (e.g., `dense_4h_to_h` in Pythia).
3. For each monitored layer ℓ, record the activation a^ℓ and gradient g^ℓ for the token position that predicts t.
4. Define the contribution score for neuron i at layer ℓ as

$$c_i^\ell \;=\; |a_i^\ell \cdot g_i^\ell|.$$

We then take the top-k neurons per layer according to c_i^ℓ. Jaccard overlap is computed over the union of layers considered. This procedure is implemented in `contribution_score.py` (Appendix A.1).

4.3 Gradient-Free Contribution via FiNE Weight Projection

The second neuron scoring method approximates FiNE's Eq. (3) [7]. For a given fact, we:

- Record the FFN input activations (before the second linear layer) q^ℓ at each layer ℓ.
- Obtain the FFN output weights W_{out}^ℓ and the unembedding matrix W_u.
- For a target token t with unembedding row $v_t = W_u[t, :]$, compute

$$\text{proj}^\ell = v_t W_{\text{out}}^\ell \in \mathbb{R}^{d_m},$$

  and define neuron scores as $c_i^\ell = q_i^\ell \cdot \text{proj}_i^\ell$ (we often take $|c_i^\ell|$).

This yields a gradient-free contribution that depends only on forward activations and static weights. Our implementation is in `contribution_score_FiNE.py` (Appendix A.2).

In the current draft, we have verified that this method runs end-to-end on all four fact pairs and can produce Jaccard overlap statistics, but we have **not yet systematically compared** the gradient-based and FiNE-style contributions on larger datasets.

4.4 ROME Editing in Mixed Precision

For layer-level editing, we implement ROME on Pythia-6.9B using the EasyEdit library, with several modifications to make it run under Colab GPU memory constraints:

- We quantize most layers to 8-bit (via `bitsandbytes`), except for the target FFN layer (layer 28), which remains in higher precision so that the rank-one update remains well-behaved.
- We patch EasyEdit's `compute_v.py` to fix gradient and indexing issues encountered on GPT-NeoX models.
- For each fact pair, we reload the model fresh, apply a single ROME edit, and then measure pre- and post-edit token probabilities on Fact A and Fact B.

Our full script (`ROME.py`) is included in Appendix A.3.

4.5 FiNE Editing Framework (Implemented, Experiments Pending)

We implement a FiNE-style neuron-level editing framework in several modules:

Neuron localization. The function `compute_fine_contributions` (in `editing_methods.py`) implements Eq. (3) from FiNE: for each fact, layer, and neuron, it calculates

$$c(i, \ell, t) = q_i^\ell \cdot (W_u W_{\text{out}}^\ell)_{t,i},$$

using the FiNE-style projection. The helper `locate_neurons_fine` (in `fine_editing.py`) then follows Algorithm 1 in FiNE, selecting top-k neurons per token and forming a union U_j for each fact.

Neuron patch parameterization. We introduce a `FinePatch` module that attaches forward hooks to the FFN output layers. For each selected neuron (ℓ, i) we maintain a vector $z_{(\ell,i)}$, and modify FFN outputs with a low-rank adjustment that is equivalent to changing row i of W_{out}^ℓ. This matches the parameterization described in Section 4.2 of FiNE.

Optimization objective. Our `fine_loss_step` implements FiNE's objective:

$$\begin{aligned}
L_{\text{edit}} &= -\log P_{M'}(o^*|p), \\
L_{\text{old}} &= -\log P_{M'}(o|p), \\
L_{\text{pen}} &= \max(0, \log P_M(o|p) - \log P_{M'}(o|p)), \\
L_{\text{total}} &= L_{\text{edit}} + \alpha_{\text{kl}} \cdot \text{KL}(P_M \| P_{M'}) + \beta_{\text{pen}} L_{\text{pen}} + \gamma \|Z\|_2^2.
\end{aligned}$$

The main entry point `apply_fine_edit` orchestrates localization, patch construction, and gradient descent on Z. Hyperparameters are stored in `FineConfig`.

Status. All of the above components compile and run on small examples, but we have **not yet completed** large-scale FiNE editing experiments on Pythia-2.8B or Pythia-6.9B due to time and compute. In particular, the results for experiments E2a/E3a (locality on SET-POLY and SET-MONO) are still in progress and are **not reported numerically in this draft**.

4.6 Experiment Registry and Runner

We introduce a simple experiment registry (`experiment_registry.py`) and main driver (`run_experiment.py`) with the following experiment codes:

- **100: E1 – Overlap Dataset.** Build SET-POLY and SET-MONO by computing neuron overlaps and Jaccard thresholds.
- **210: E2a – Locality on SET-POLY.** Compare FiNE vs ROME edits on high-overlap pairs. **(Not yet run for FiNE.)**
- **300: E3a – Locality on SET-MONO.** Same as above but on low-overlap pairs. **(Not yet run for FiNE.)**
- **410, 500:** Ablations and global locality skeletons (not yet implemented).

The following command (also included in `run.txt`) constructs the overlap dataset:

```
python run_experiment.py --exp_code 100 --model_name EleutherAI/pythia-6.9b \
    --facts_path data/facts.json --out_dir results/overlap_pythia28
```

5 Experiments

5.1 Setup

For our preliminary experiments we focus on:

- **Model:** Pythia-6.9B, quantized to 8-bit with the target editing layer in higher precision.

Table 1: ROME locality behavior on four fact pairs. Editing Fact A often causes significant degradation on Fact B, especially in high-overlap cases such as (France, Paris).

Pair ID	Fact A Edit	Fact B Target	P_B (%)	P'_B (%)	Drop (%)	Conclusion
Pair 3 (Paris)	France→Rome	Eiffel Tower→Paris	35.74	10.14	71.6	Severe collapse
Pair 4 (Language)	Spanish→Bonjour	Hola→Spanish	62.55	41.65	33.4	Moderate collapse
Pair 1 (Jobs)	Jobs→Microsoft	Jobs→Pixar	2.10	0.05	97.5	Complete erasure
Pair 2 (Gates)	Gates→Apple	Gates→Windows	0.00	0.00	–	Model blind spot

- **Target layer:** Layer 28, selected based on earlier neuron overlap analysis showing relatively high Jaccard overlap for the France/Paris pair at that layer.
- **Dataset:** The four fact pairs described in Section 3.
- **Hardware:** Google Colab Pro with A100 GPU (40GB), CPU offloading for some parameters.

For each pair, we perform a single ROME edit that changes the object of Fact A (e.g., France→Rome) and then measure:

- The probability of the *new* object for Fact A after editing.
- The probability of the *correct* object for Fact B before and after editing.

5.2 Layer-Level Edits with ROME

Table 1 summarizes our ROME experiments on the four fact pairs. P_B and P'_B denote the probability of the correct Fact B answer before and after the edit, respectively. "Drop" is the relative percentage drop.

These results support the intuition that when two facts share substantial internal representation (as in the France/Paris pair), rank-one layer-level edits can strongly interfere with related knowledge. In particular, editing "France→Rome" nearly destroys the model's ability to answer "Eiffel Tower is located in Paris."

5.3 Planned FiNE Neuron-Level Editing Experiments (Not Yet Completed)

Our main scientific question is how neuron-level FiNE edits compare to layer-level ROME edits in terms of locality, especially on SET-POLY vs. SET-MONO.

We have implemented the full FiNE pipeline (Section 4.5) and the evaluation metrics (Section 6) in code. Our **planned** experiments include:

- **E1 (code 100):** Use FiNE neuron localization to build SET-POLY and SET-MONO from an extended factual dataset (e.g., a subset of CounterFact). This experiment has been partially implemented, but we have not yet finalized the thresholds or the resulting datasets.
- **E2a (code 210):** On SET-POLY, compare FiNE vs. ROME in terms of edit success, locality, and over-editing rate. Hypothesis: FiNE should achieve better locality than ROME on high-overlap pairs.
- **E3a (code 300):** On SET-MONO, compare FiNE vs. ROME in the low-overlap regime. Hypothesis: both methods should perform well, but FiNE may be overkill.

Status: At the time of this draft, we have not yet run E2a/E3a end-to-end on Pythia-2.8B or Pythia-6.9B. No FiNE quantitative results are reported in this version of the paper. We only describe the experimental design and implementation, and leave the actual execution and analysis for the final report.

6 Evaluation Implementation

To make our analysis reproducible and align with FiNE and KnowEdit-style evaluation, we implemented an evaluation module (`evaluation.py`) that provides:

- `FactResult`: captures the prompt, expected answer, correctness, average log-probability, and generated response.
- `EditPairResult`: stores pre- and post-edit results for Fact A and Fact B along with method metadata.
- Functions to compute edit success, locality, over-editing rate, and fluency (bigram + trigram entropy).

These implementations match the definitions described in Section 3 and are ready to be applied to both ROME and FiNE experiments once the latter are executed.

7 Discussion and Limitations

Our preliminary ROME experiments confirm that layer-level rank-one updates can significantly disrupt related knowledge, particularly when neuron-level overlaps are high. This aligns with the polysemanticity picture: when many neurons are shared between two facts, changing their outgoing weights to accommodate one fact inevitably changes the other.

However, several limitations remain:

- **FiNE experiments are incomplete.** The most important limitation of this draft is that we have not yet run the FiNE editing experiments. As a result, we cannot yet draw conclusions about the relative merits of neuron-level versus layer-level editing.
- **Small and hand-crafted dataset.** Our current fact pairs are small and manually constructed. We plan to use an extended factual dataset (e.g., CounterFact-like) to build SET-POLY and SET-MONO in a more systematic way.
- **Limited metrics for ROME.** For ROME, we only report pre-/post-edit probabilities on Fact B. In the final report, we aim to integrate our full evaluation pipeline (including fluency and over-editing) for both ROME and FiNE.

8 Conclusion and Future Work

In this draft, we set up an experimental framework to study how polysemanticity affects the locality of model editing, contrasting neuron-level (FiNE-style) and layer-level (ROME) interventions. We implemented:

- Two neuron contribution scoring methods (activation–gradient and FiNE weight projection).
- A FiNE-style neuron patch parameterization and optimization objective.
- A mixed-precision ROME implementation on Pythia-6.9B.
- An evaluation pipeline for edit success, locality, over-editing, and fluency.

Our initial ROME experiments show substantial interference on high-overlap fact pairs such as France/Paris, suggesting that polysemanticity indeed constrains locality at the layer level. The key missing piece is the FiNE neuron-level editing experiments, which we plan to complete for the final report. We expect these results to clarify whether neuron-level control can mitigate the costs of superposition in practice.

Reproducibility and Code Availability

To satisfy the course reproducibility requirement, we release all code used in this project, including analysis scripts, editing implementations, and experiment runners, at the following public GitHub repository:

https://github.com/pekingappG/CS182-Project/tree/main?tab=readme-ov-file

The repository includes instructions and scripts to reproduce all tables and figures in this report (and additional FiNE experiments in the final version), assuming sufficient GPU resources.

Acknowledgments

We thank the CS 182 course staff for helpful discussions and for providing guidance on project selection and evaluation.

References

[1] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022.

[2] Eric Mitchell, Charles Lin, Antoine Bosselut, Christopher D. Manning, and Chelsea Finn. Fast Model Editing at Scale. In *International Conference on Learning Representations*, 2022.

[3] Kevin Meng, et al. Mass Editing Memory in a Transformer. In *International Conference on Learning Representations*, 2023.

[4] Thomas Hartvigsen, et al. Tuning Language Models by Proxy: An Investigation into Parameter-Efficient Model Editing. In *ACL*, 2023.

[5] Neil Elhage, et al. Toy Models of Superposition. Anthropic, 2022.

[6] Arthur Conmy, et al. Finding and Editing Multi-Modal Neurons. In *ACL*, 2024.

[7] Xinyun Zheng, et al. Precise Localization of Memories in Large Language Models. In *International Conference on Learning Representations*, 2025.

A Additional Implementation Details

This appendix summarizes the most relevant code components and generated outputs. Full source code is available in our GitHub repository.

A.1 Activation–Gradient Contribution Scores

Listing 1 shows the core implementation of the activation–gradient contribution score used to compute neuron overlap for the four fact pairs on Pythia-160M.

Listing 1: Activation–gradient neuron contribution scores (abridged).

```
import torch
import numpy as np
from transformers import GPTNeoXForCausalLM, AutoTokenizer

print("...")
model_name = "EleutherAI/pythia-160m"
device = "cuda" if torch.cuda.is_available() else "cpu"

model = GPTNeoXForCausalLM.from_pretrained(model_name).to(device)
tokenizer = AutoTokenizer.from_pretrained(model_name)
print(f":␣{model_name},␣:␣{device}")

def get_neuron_scores(model, prompt, target_word, layer_indices):
   input_text = prompt + target_word
   inputs = tokenizer(input_text, return_tensors="pt").to(device)
   target_ids = inputs["input_ids"].clone()

   activations = {}
   gradients = {}
   handles = []

   def get_act_hook(layer_idx):
      def hook(module, input, output):
         # use -2 because last token predicts target
         activations[layer_idx] = output[:, -2, :].detach()
      return hook

   def get_grad_hook(layer_idx):
      def hook(module, grad_input, grad_output):
         gradients[layer_idx] = grad_output[0][:, -2, :].detach()
      return hook

   for layer_idx in layer_indices:
      target_layer = model.gpt_neox.layers[layer_idx].mlp.dense_4h_to_h
      handles.append(target_layer.register_forward_hook(get_act_hook(
          layer_idx)))
      handles.append(target_layer.register_full_backward_hook(
          get_grad_hook(layer_idx)))

   model.zero_grad()
   outputs = model(**inputs, labels=target_ids)
   loss = outputs.loss
   loss.backward()

   scores = {}
   for layer_idx in layer_indices:
      act = activations[layer_idx].squeeze()
      grad = gradients[layer_idx].squeeze()
      score = torch.abs(act * grad)
      scores[layer_idx] = score

   for h in handles:
```

```
        h.remove()

    return scores
```

A.2 Gradient-Free FiNE-Style Contribution Scores

Listing 2 illustrates our gradient-free FiNE-style contribution computation, which uses $W_u W_{\text{out}}^{\ell}$ and forward activations.

Listing 2: Gradient-free FiNE-style neuron contributions (abridged).

```
import torch
from transformers import GPTNeoXForCausalLM, AutoTokenizer

def get_gradient_free_contribution_scores(model, tokenizer, prompt,
    target, layer_indices):
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    target_token_id = tokenizer.encode(target)[0]
    print(f"Calculating␣contribution␣for␣target␣token:␣'{target}'␣(ID:␣{
        target_token_id})")

    activations = {}
    handles = []

    def get_activation_hook(layer_idx):
        def hook(module, args, output):
            # args[0] is FFN hidden activations before dense_4h_to_h
            activations[layer_idx] = args[0][:, -1, :].detach()
        return hook

    for i in layer_indices:
        layer_module = model.gpt_neox.layers[i].mlp.dense_4h_to_h
        handles.append(layer_module.register_forward_hook(
            get_activation_hook(i)))

    with torch.no_grad():
        model(**inputs)

    w_u = model.embed_out.weight
    target_vector = w_u[target_token_id, :]
    scores = {}

    print("Computing␣scores␣using␣Weight␣Projection...")
    for i in layer_indices:
        act = activations[i].squeeze()
        w_out = model.gpt_neox.layers[i].mlp.dense_4h_to_h.weight
        neuron_projection = torch.matmul(target_vector, w_out)
        layer_scores = act * neuron_projection
        scores[i] = layer_scores

    for h in handles:
        h.remove()

    return scores
```

A.3 ROME Implementation for Pythia-6.9B

Our ROME implementation builds on EasyEdit and adapts it to Pythia-6.9B with mixed-precision loading. Listing 3 shows an abridged version of the script focusing on: (1) patching `compute_v.py`, (2) loading the model with 8-bit quantization except for the target layer, and (3) computing pre- and post-edit probabilities for each fact pair.

Listing 3: Abridged ROME implementation for Pythia-6.9B.

```
import os, sys, gc
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer,
    BitsAndBytesConfig
from easyeditor import ROMEHyperParams, apply_rome_to_model

MODEL_NAME = "EleutherAI/pythia-6.9b"
EDIT_LAYER = 28
LOSS_LAYER = 31

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
if tokenizer.pad_token is None:
   tokenizer.pad_token = tokenizer.eos_token

hparams = ROMEHyperParams(
   alg_name="ROME",
   model_name=MODEL_NAME,
   device=0,
   layers=[EDIT_LAYER],
   rewrite_module_tmp="gpt_neox.layers.{}.mlp.dense_4h_to_h",
   layer_module_tmp="gpt_neox.layers.{}",
   mlp_module_tmp="gpt_neox.layers.{}.mlp",
   attn_module_tmp="gpt_neox.layers.{}.attention",
   ln_f_module="gpt_neox.final_layer_norm",
   lm_head_module="embed_out",
   fact_token="subject_last",
   v_num_grad_steps=20,
   v_lr=5e-1,
   v_loss_layer=LOSS_LAYER,
   v_weight_decay=1e-3,
   clamp_norm_factor=4,
   kl_factor=0.0625,
   mom2_dataset="wikipedia",
   mom2_n_samples=100,
   mom2_dtype="float32",
   stats_dir="./data/stats",
)

FACT_PAIRS = [
   {
      "id": "Pair␣3␣(Paris)",
      "prompt_a": "The␣capital␣of␣France␣is",
      "subject": "France",
      "target_true": "␣Paris",
      "target_new": "␣Rome",
      "prompt_b": "The␣Eiffel␣Tower␣is␣located␣in",
      "target_b_true": "␣Paris"
   },
   # other pairs omitted for brevity
]

def get_prob(model, prompt, target_word):
   inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
   try:
      target_id = tokenizer("␣" + target_word.strip())["input_ids"][0]
   except:
      target_id = tokenizer(target_word.strip())["input_ids"][0]
   with torch.no_grad():
      outputs = model(**inputs)
   probs = torch.nn.functional.softmax(outputs.logits[0, -1, :], dim=-1)
   return probs[target_id].item()

def main():
```

```
    quant_config = BitsAndBytesConfig(
        load_in_8bit=True,
        llm_int8_threshold=6.0,
        llm_int8_skip_modules=[f"gpt_neox.layers.{EDIT_LAYER}.mlp.
            dense_4h_to_h"],
        llm_int8_enable_fp32_cpu_offload=True,
    )

    for pair in FACT_PAIRS:
        model = AutoModelForCausalLM.from_pretrained(
            MODEL_NAME,
            quantization_config=quant_config,
            device_map="auto"
        )

        prob_a_pre = get_prob(model, pair["prompt_a"], pair["target_true"])
        prob_b_pre = get_prob(model, pair["prompt_b"], pair["target_b_true"
            ])

        requests = [{
            "prompt": pair["prompt_a"],
            "subject": pair["subject"],
            "target_new": pair["target_new"]
        }]

        torch.cuda.empty_cache()
        gc.collect()

        metrics, _ = apply_rome_to_model(
            model,
            tokenizer,
            requests,
            hparams,
            copy=False,
            return_orig_weights=False
        )

        prob_a_post_new = get_prob(model, pair["prompt_a"], pair["
            target_new"])
        prob_b_post = get_prob(model, pair["prompt_b"], pair["target_b_true
            "])

        print(f"{pair['id']}␣|␣A␣new:␣{prob_a_post_new:.4f}␣|␣"
              f"B␣pre:␣{prob_b_pre:.4f}␣->␣post:␣{prob_b_post:.4f}")

        del model
        torch.cuda.empty_cache()
        gc.collect()

if __name__ == "__main__":
    main()
```

A.4 FiNE Editing and Evaluation Modules

For brevity, we do not reproduce the entire `fine_editing.py`, `editing_methods.py`, `evaluation.py`, and `run_experiment.py` here. These files implement:

- FiNE neuron localization (Eq. (3)),
- neuron patch parameterization and optimization objective (Eq. (4)–(7)),
- experiment registry (codes 100/210/300/410/500),
- and the full evaluation pipeline for edit success, locality, over-editing, and fluency.

They are available in full in our public GitHub repository, and directly correspond to the descriptions in Sections 4.5 and 6.